

Python Data Structures

A Complete Reference: **set**, **frozenset** & **dict**

Methods • Time Complexity • Space Complexity • Comparisons

Table of Contents

- 1. Overview of Set, Frozenset and Dict
- 2. The set type — full method reference
- 3. Set operators & algebra (union, intersection, etc.)
- 4. The frozenset type
- 5. The dict type — full method reference
- 6. Dict view objects (keys, values, items)
- 7. Side-by-side comparison: set vs dict vs list vs tuple
- 8. Hashability rules
- 9. When to use which structure
- 10. Common pitfalls

1. Overview

set

An unordered collection of unique, hashable elements. Implemented internally as a hash table (like a dict with keys only, no values). Mutable: elements can be added or removed after creation.

frozenset

The immutable twin of set. Same content behavior, but cannot be changed after creation. Because it's immutable, it is itself hashable — so it can be used as a dict key or placed inside another set.

dict

An unordered (technically insertion-ordered since Python 3.7) mapping of unique, hashable keys to values. Also implemented as a hash table. Mutable: keys/values can be added, updated, or removed.

Quick creation syntax

```
s = set() # empty set (NOT {})  
s = {1, 2, 3} # set literal  
fs = frozenset([1,2,3]) # frozenset  
d = {} # empty dict  
d = {'a': 1, 'b': 2} # dict literal
```

Note: `{}` always creates an empty **dict**, never an empty set. This is one of the most common beginner mistakes in Python.

2. The set Type — Full Method Reference

2.1 Mutating methods (change the set in place)

Method	Description	Avg Time	Worst Time	Space
s.add(x)	Add a single element x	O(1)	O(n)	O(1)
s.remove(x)	Remove x; raises KeyError if absent	O(1)	O(n)	O(1)
s.discard(x)	Remove x if present; no error if absent	O(1)	O(n)	O(1)
s.pop()	Remove & return an arbitrary element	O(1)	O(n)	O(1)
s.clear()	Remove all elements	O(n)	O(n)	O(1)
s.update(*iterables)	Add all elements from other iterables (union in place)	O(sum of lens)	O(sum of lens)	O(k)
s.intersection_update(*it)	Keep only elements found in all iterables	O(len(s))	O(len(s)*m)	O(1)
s.difference_update(*it)	Remove elements found in other iterables	O(sum of lens)	O(sum of lens)	O(1)
s.symmetric_difference_update(o)	Keep elements in exactly one of the two sets	O(len(s)+len(o))	O(len(s)+len(o))	O(len(o))

2.2 Non-mutating methods (return new values, don't change s)

Method	Description	Time	Space
s.union(*it)	New set with elements from s and all others	O(sum of lens)	O(sum of lens)
s.intersection(*it)	New set with elements common to all	O(min len)	O(min len)
s.difference(*it)	New set with elements in s not in others	O(len(s))	O(len(s))
s.symmetric_difference(o)	New set: elements in exactly one of the two	O(len(s)+len(o))	O(len(s)+len(o))
s.copy()	Shallow copy of the set	O(n)	O(n)
s.isdisjoint(o)	True if s and o share no elements	O(min(len(s),len(o)))	O(1)
s.issubset(o) / s <= o	True if every element of s is in o	O(len(s))	O(1)
s.issuperset(o) / s >= o	True if s contains every element of o	O(len(o))	O(1)

2.3 Built-in functions / operators usable on sets

Expression	Description	Time
x in s	Membership test	O(1) avg, O(n) worst
len(s)	Number of elements	O(1)
for x in s	Iterate all elements	O(n)
bool(s)	False if empty, True otherwise	O(1)
s == o / s != o	Equality by content, not order	O(min(len(s),len(o)))

All set elements must be hashable. You cannot put a list, dict, or another set inside a set — but tuples and frozensets are allowed since they are hashable.

3. Set Operators & Algebra

Every set-algebra method above has an equivalent operator. Operators require both operands to be actual sets/frozensets; the method form accepts any iterable.

Operation	Operator	Method Equivalent	Time
Union	$s1 \mid s2$	<code>s1.union(s2)</code>	$O(\text{len}(s1) + \text{len}(s2))$
Intersection	$s1 \& s2$	<code>s1.intersection(s2)</code>	$O(\min(\text{len}(s1), \text{len}(s2)))$
Difference	$s1 - s2$	<code>s1.difference(s2)</code>	$O(\text{len}(s1))$
Symmetric difference	$s1 \wedge s2$	<code>s1.symmetric_difference(s2)</code>	$O(\text{len}(s1) + \text{len}(s2))$
In-place union	$s1 \mid= s2$	<code>s1.update(s2)</code>	$O(\text{len}(s2))$
In-place intersection	$s1 \&= s2$	<code>s1.intersection_update(s2)</code>	$O(\text{len}(s1))$
In-place difference	$s1 -= s2$	<code>s1.difference_update(s2)</code>	$O(\text{len}(s2))$
Subset	$s1 \<= s2$	<code>s1.issubset(s2)</code>	$O(\text{len}(s1))$
Proper subset	$s1 \< s2$	—	$O(\text{len}(s1))$
Superset	$s1 \>= s2$	<code>s1.issuperset(s2)</code>	$O(\text{len}(s2))$

Example

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
a | b # {1, 2, 3, 4, 5, 6}
a & b # {3, 4}
a - b # {1, 2}
a ^ b # {1, 2, 5, 6}
```

4. The frozenset Type

frozenset supports every **non-mutating** method and operator listed above (union, intersection, difference, symmetric_difference, copy, isdisjoint, issubset, issuperset, in, len, iteration, equality) with identical time complexity. It does NOT support any mutating method.

Feature	set	frozenset
Mutable	Yes	No
Hashable	No	Yes
Can be a dict key	No	Yes
Can be an element of another set	No	Yes
.add() / .remove() / .discard() / .pop() / .clear()	Yes	No (AttributeError)
.update() / .intersection_update() / etc.	Yes	No (AttributeError)
union / intersection / difference / etc.	Yes	Yes
Typical use	Building/mutating a collection	Fixed set used as a key or nested element

```
d = {frozenset([1,2]): 'pair A', frozenset([3,4]): 'pair B'} # works s = {frozenset([1,2]),  
frozenset([3,4])} # works d = {{1,2}: 'x'} # TypeError: unhashable type: 'set'
```

5. The dict Type — Full Method Reference

5.1 Access & lookup

Method / Expression	Description	Avg Time	Worst Time
d[key]	Get value; raises KeyError if missing	O(1)	O(n)
d.get(key, default=None)	Get value or default, never raises	O(1)	O(n)
d.setdefault(key, default)	Get value; if missing, insert default and return it	O(1)	O(n)
key in d	Membership test on keys	O(1)	O(n)
len(d)	Number of key-value pairs	O(1)	O(1)

5.2 Mutating methods

Method	Description	Avg Time	Worst Time
d[key] = value	Insert or overwrite a key's value	O(1)	O(n)
d.update(other)	Merge keys/values from another dict or iterable of pairs	O(len(other))	O(len(other))
d.pop(key, default)	Remove key, return its value (or default)	O(1)	O(n)
d.popitem()	Remove & return the last inserted (key,value) pair	O(1)	O(1)
del d[key]	Delete a key (raises KeyError if missing)	O(1)	O(n)
d.clear()	Remove all key-value pairs	O(n)	O(n)
d.setdefault(key, default)	Insert default only if key missing	O(1)	O(n)

5.3 Views & iteration

Method	Description	Time
d.keys()	View of all keys (live, dynamic)	O(1) to create, O(n) to iterate
d.values()	View of all values (live, dynamic)	O(1) to create, O(n) to iterate
d.items()	View of (key, value) pairs (live, dynamic)	O(1) to create, O(n) to iterate
d.copy()	Shallow copy of the dict	O(n)
for k in d	Iterate over keys (default)	O(n)

5.4 Class / static methods

Method	Description	Time
dict.fromkeys(iterable, value=None)	New dict with keys from iterable, all mapped to value	O(n)

Since Python 3.7, dicts preserve insertion order as a language guarantee (it was a CPython implementation detail in 3.6). This does not change any time complexity above.

6. Dict View Objects in Detail

`keys()`, `values()` and `items()` return **view objects**, not lists. They are dynamic: if the dict changes, the view reflects the change immediately without needing to be recreated.

```
d = {'a': 1, 'b': 2} keys_view = d.keys() print(keys_view) # dict_keys(['a', 'b']) d['c'] = 3
print(keys_view) # dict_keys(['a', 'b', 'c']) <- updated automatically # keys() and items() views
support set-like operations: d1 = {'a': 1, 'b': 2} d2 = {'b': 3, 'c': 4} d1.keys() & d2.keys() #
{'b'} (intersection) d1.keys() | d2.keys() # {'a', 'b', 'c'} (union)
```

View	Set-like?	Why
<code>d.keys()</code>	Yes — supports <code>&</code> , <code> </code> , <code>-</code> , <code>^</code>	Keys are always unique, like set elements
<code>d.items()</code>	Yes, if values are hashable	(key,value) pairs are unique tuples
<code>d.values()</code>	No	Values can repeat — not a valid set

7. Side-by-Side Comparison

Feature	list	tuple	set	dict
Ordered	Yes	Yes	No	Yes (insertion, 3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates allowed	Yes	Yes	No	No (keys)
Indexable (<code>x[i]</code>)	Yes	Yes	No	By key only
Hashable	No	Yes*	No	No
Lookup by value: <code>x in s</code>	$O(n)$	$O(n)$	$O(1)$ avg	$O(1)$ avg (keys)
Insert	$O(1)$ append / $O(n)$ middle	N/A (immutable)	$O(1)$ avg	$O(1)$ avg
Delete	$O(n)$	N/A	$O(1)$ avg	$O(1)$ avg
Typical literal	<code>[1,2,3]</code>	<code>(1,2,3)</code>	<code>{1,2,3}</code>	<code>{'a':1}</code>
Empty literal	<code>[]</code>	<code>()</code>	<code>set()</code>	<code>{}</code>

*tuple is hashable only if every element inside it is also hashable.

Big-picture takeaway

Need...	Use
Fast membership tests, no duplicates, no order	set
A fixed collection to use as a dict key or set element	frozenset
Key & value lookups	dict
Ordered, index-based, allows duplicates	list
Fixed, ordered, allows duplicates (and hashable if contents allow)	tuple

8. Hashability Rules

An object is **hashable** if it has a fixed hash value for its entire lifetime (via `__hash__()`) and defines equality consistently (equal objects → equal hashes). Hashability is required for:

- Set elements
- Dict keys

Type	Hashable?	Reason
int, float, bool, str	Yes	Immutable
tuple	Yes, if all elements are hashable	Immutable container
frozenset	Yes	Immutable
list	No	Mutable
dict	No	Mutable
set	No	Mutable
Custom object (default)	Yes	Hashes by id() unless <code>__eq__</code> overridden

```
hash((1, 2, 3)) # OK – works hash((1, [2, 3])) # TypeError – contains a mutable list hash({1, 2, 3}) # TypeError – set is mutable hash(frozenset([1,2,3])) # OK – frozenset is immutable
```

9. When to Use Which Structure

Scenario	Best Choice	Why
Remove duplicates from a list	set	set(my_list) drops dupes in O(n)
Check 'have I seen this before?' repeatedly	set	O(1) membership vs O(n) for list
Count occurrences of items	dict (or collections.Counter)	Key → count mapping
Group items by a category	dict of lists	Key → list of matching items
Need a fixed group of items as a dict key	frozenset	Only hashable option for grouped-unique values
Preserve order and allow duplicates	list	Sets and dicts remove dupes / dicts key by uniqueness
Fast intersection/union of two collections	set	Built-in O(n)-class set algebra
Fixed, unchangeable sequence	tuple	Immutability signals intent and enables hashing

10. Common Pitfalls

Pitfall	Explanation / Fix
<code>{}</code> makes a dict, not a set	Use <code>set()</code> for an empty set.
Adding a list to a set/dict key	Lists are unhashable. Convert to <code>tuple()</code> first if needed.
Assuming set/dict preserve insertion order	Set/dict have NO order guarantee, unlike dicts (3.7+).
Mutating a dict while iterating over it	Raises <code>RuntimeError</code> . Iterate over <code>list(d.items())</code> instead.
Using <code>d[key]</code> instead of <code>d.get(key)</code>	<code>d[key]</code> raises <code>KeyError</code> on a missing key; <code>.get()</code> returns <code>None</code> or a default safely.
Forgetting <code>.add()</code> only takes ONE argument	Use <code>.update()</code> to add multiple elements at once.
Overriding <code>__eq__</code> without <code>__hash__</code>	Makes custom objects unhashable by default — define both together.
<code>.remove()</code> vs <code>.discard()</code>	<code>.remove()</code> raises <code>KeyError</code> if the element is absent; <code>.discard()</code> silently does nothing.

End of reference. All complexities assume CPython's standard hash table implementation; worst-case $O(n)$ scenarios (hash collisions) are rare in practice with well-distributed hash functions.